

Implementation of Finite Elements

This lecture shows how to implement our own finite elements in C++, and how to use them within the NGSolve language. We implement first order and second order triangular finite elements.

The finite element is implemented in its own package on github: [TUWien-ASC/NGS-myfe](https://github.com/TUWien-ASC/NGS-myfe)

- basis functions are implemented in myelement.cpp/hpp
- the transformation from reference-element to physical elements in in mydiffop.hpp
- the connectivity is implemented in the `FESpace` in myfespace.cpp/hpp
- the Python-bindings are in mymodule.cpp

```
from ngsolve import *
from ngsolve.webgui import Draw
from myfe import *
mesh = Mesh(unit_square.GenerateMesh(maxh=0.2, quad_dominated=False))
```

Loading myfe library

We can now create an instance of our own finite element space:

```
fes = MyFESpace(mesh, secondorder=True, dirichlet="left|bottom|top")
```

```
Constructor of MyFESpace
Flags = secondorder
dirichlet = 0: 1
1: 3
2: 4
```

[Back to top](#)

You have chosen second order elements
Update MyFESpace, #vert = 38, #edge = 91

and use it within NGSolve such as the builtin finite element spaces:

```
print ("ndof = ", fes.ndof)
```

```
ndof = 129
```

```
gfu = GridFunction(fes)
gfu.Set(x*y)

Draw (gfu)
Draw (grad(gfu)[0], mesh);
```

Open Controls

-5.4e-17 0.500 1.00

Y
Z X

NGSolve 6.2.2603-28-gdfd0ea93c

[Back to top](#)

[Open Controls](#)

-1.1e-15 0.500 1.00

Y
Z X

NGSolve 6.2.2603-28-gdfd0ea93c

and solve the standard problem:

```
u,v = fes.TnT()
a = BilinearForm(grad(u)*grad(v)*dx).Assemble()
f = LinearForm(1*v*dx).Assemble()
gfu.vec.data = a.mat.Inverse(fes.FreeDofs())*f.vec
Draw (gfu);
```

Print to PDF ►

[Back to top](#)

[Open Controls](#)

0.00

0.0569

0.114

Y
Z X

NGSolve 6.2.2603-28-gdfd0ea93c

Draw basis functions:

```
gfu.vec[:] = 0  
gfu.vec[mesh.nv-3] = 1  
gfu.vec[fes.ndof-1] = 1  
Draw (gfu, order=2);
```

[Back to top](#)

[Open Controls](#)

0.00

0.500

1.00

Z Y
X

NGSolve 6.2.2603-28-gdfd0ea93c

Documentation provided in the DocInfo structure is available in Python - help. Look for 'secondorder'.

[help \(MyFESpace\)](#)[Back to top](#)

Help on class MyFESpace in module myfe:

```
class MyFESpace(ngsolve.comp.FESpace)
|   My own FESpace.
|
|   My own FESpace provides first and second order triangular elements.
|
|   Keyword arguments can be:
|
|   order: int = 1
|       order of finite element space
|   complex: bool = False
|       Set if FESpace should be complex
|   dirichlet: regexpr
|       Regular expression string defining the dirichlet boundary.
|       More than one boundary can be combined by the | operator,
|       i.e.: dirichlet = 'top|right'
|   dirichlet_bbnd: regexpr
|       Regular expression string defining the dirichlet bboundary,
|       i.e. points in 2D and edges in 3D.
|       More than one boundary can be combined by the | operator,
|       i.e.: dirichlet_bbnd = 'top|right'
|   dirichlet_bbbnd: regexpr
|       Regular expression string defining the dirichlet bbboundary,
|       i.e. points in 3D.
|       More than one boundary can be combined by the | operator,
|       i.e.: dirichlet_bbbnd = 'top|right'
|   definedon: Region or regexpr
|       FESpace is only defined on specific Region, created with mesh.Materials('rege:
|       or mesh.Boundaries('regexpr'). If given a regexpr, the region is assumed to be
|       mesh.Materials('regexpr').
|   dim: int = 1
|       Create multi dimensional FESpace (i.e. [H1]^3)
|   dgjumps: bool = False
|       Enable discontinuous space for DG methods, this flag is needed for DG methods
|       since the dofs have a different coupling then and this changes the sparsity
|       pattern of matrices.
|   autoupdate: bool = False
|       Automatically update on a change to the mesh.
|   low_order_space: bool = True
|       Generate a lowest order space together with the high-order space,
|       needed for some preconditioners.
|   hoprolongation: bool = False
|       Create high order prolongation operators,
|       only available for H1 and L2 on simplicial meshes
|   order_policy: ORDER_POLICY = ORDER_POLICY.OLDSTYLE
|       CONSTANT .. use the same fixed order for all elements,
|       NODAL ..... use the same order for nodes of same shape,
|       VARIABLE ... use an individual order for each edge, face and cell,
|       OLDSTYLE .. as it used to be. Back to top st decade
|   print: bool = False
|       (historic) print some output into file set by 'SetTestoutFile'
|   secondorder: bool = False
|       Use second order basis functions
```

```

Method resolution order:
  MyFESpace
  ngsolve.comp.FESpace
  ngsolve.comp.NGS_Object
  pybind11_builtins.pybind11_object
  builtins.object

Methods defined here:

__getstate__(...)
    __getstate__(self: ngsolve.comp.FESpace, /) -> tuple

__init__(...)
    __init__(self: myfe.MyFESpace, mesh: ngsolve.comp.Mesh, **kwargs) -> None

__setstate__(...)
    __setstate__(self: myfe.MyFESpace, arg0: tuple) -> None

-----
Static methods defined here:

__flags_doc__(...) method of pybind11_builtins.pybind11_detail_function_record_
    __flags_doc__() -> dict

-----
Data descriptors defined here:

__dict__

-----
Methods inherited from ngsolve.comp.FESpace:

ApplyM(...)
    ApplyM(self: ngsolve.comp.FESpace, vec: ngsolve.la.BaseVector, rho: ngsolve

    Apply mass-matrix. Available only for L2-like spaces

ConvertL2Operator(...)
    ConvertL2Operator(self: ngsolve.comp.FESpace, l2space: ngsolve.comp.FESpace

CouplingType(...)
    CouplingType(self: ngsolve.comp.FESpace, dofnr: typing.SupportsInt | typing

        Get coupling type of a degree of freedom.

    Parameters:

        dofnr : int
            input dof number
            Back to top

CreateDirectSolverCluster(...)
    CreateDirectSolverCluster(self: ngsolve.comp.FESpace, **kwargs) -> list

```

```

CreateSmoothingBlocks(...)
    CreateSmoothingBlocks(self: ngsolve.comp.FESpace, **kwargs) -> pyngcore.pyng

    Create table of smoothing blocks for block-Jacobi/block-Gauss-Seidel preconditioning

    Every table entry describes the set of dofs belonging a Jacobi/Gauss-Seidel block

    Parameters:

    blocktype: string | [ string ] | int
        describes blocktype.
        string form ["vertex", "edge", "face", "facet", "vertexedge", ....]
        or list of strings for combining multiple blocktypes
        int is for backward compatibility with old style blocktypes

    condense: bool = False
        exclude dofs eliminated by static condensation

Elements(...)
    Elements(*args, **kwargs)
    Overloaded function.

    1. Elements(self: ngsolve.comp.FESpace, VOL_or_BND: ngsolve.comp.VorB = <VorB>) -> Iterable

    Returns an iterable range of elements.

    Parameters:

    VOL_or_BND : ngsolve.comp.VorB
        input VOL, BND, BBND,...

    2. Elements(self: ngsolve.comp.FESpace, arg0: ngsolve.comp.Region) -> collection

FinalizeUpdate(...)
    FinalizeUpdate(self: ngsolve.comp.FESpace) -> None

    finalize update

FreeDofs(...)
    FreeDofs(self: ngsolve.comp.FESpace, coupling: bool = False) -> pyngcore.pyng

    Return BitArray of free (non-Dirichlet) dofs\n
    coupling=False ... all free dofs including local dofs\n
    coupling=True .... only 'free' dofs\n
    Back to top

    Parameters:

    coupling : bool
        input coupling

```

```

    input coupling

GetDefinedOnRegion(...)
    GetDefinedOnRegion(self: ngsolve.comp.FESpace, vb: ngsolve.comp.VorB = <VorB>)

    Return mesh-region where FESpace is defined on

GetDirichletRegion(...)
    GetDirichletRegion(self: ngsolve.comp.FESpace, vb: ngsolve.comp.VorB = <VorB>)

    Return boundary Dirichlet mesh-region of FESpace

GetDofNrs(...)
    GetDofNrs(*args, **kwargs)
    Overloaded function.

    1. GetDofNrs(self: ngsolve.comp.FESpace, ei: ngsolve.comp.ElementId) -> tuple

    Parameters:

    ei : ngsolve.comp.ElementId
        input element id

    2. GetDofNrs(self: ngsolve.comp.FESpace, ni: ngsolve.comp.NodeId) -> tuple

    Parameters:

    ni : ngsolve.comp.NodeId
        input node id

GetDofs(...)
    GetDofs(self: ngsolve.comp.FESpace, region: ngsolve.comp.Region) -> pyngcore

    Returns all degrees of freedom in given region.

    Parameters:

    region : ngsolve.comp.Region
        input region

GetFE(...)
    GetFE(self: ngsolve.comp.FESpace, ei: ngsolve.comp.ElementId) -> object

    Get the finite element tuple (id, element id). Back to top

    Parameters:

    ei : ngsolve.comp.ElementId

```

```

    el : ngsolve.comp.ElementId
    input element id

GetNDofLevel(...)
    GetNDofLevel(self: ngsolve.comp.FESpace, level: typing.SupportsInt | typing

GetOrder(...)
    GetOrder(self: ngsolve.comp.FESpace, nodeid: ngsolve.comp.NodeId) -> int

    return order of node.
    by now, only isotropic order is supported here

GetTrace(...)
    GetTrace(self: ngsolve.comp.FESpace, arg0: ngsolve.comp.FESpace, arg1: ngsol

GetTraceTrans(...)
    GetTraceTrans(self: ngsolve.comp.FESpace, arg0: ngsolve.comp.FESpace, arg1:

HideAllDofs(...)
    HideAllDofs(self: ngsolve.comp.FESpace, component: object = <ngsolve.ngstd.l

    set all visible coupling types to HIDDEN_DOFs (will be overwritten by any U

InvM(...)
    InvM(self: ngsolve.comp.FESpace, rho: ngsolve.fem.CoefficientFunction = Non

Mass(...)
    Mass(self: ngsolve.comp.FESpace, rho: ngsolve.fem.CoefficientFunction = Non

ParallelDofs(...)
    ParallelDofs(self: ngsolve.comp.FESpace) -> ngsolve.la.ParallelDofs

    Return dof-identification for MPI-distributed meshes

Prolongation(...)
    Prolongation(self: ngsolve.comp.FESpace) -> ngmg::Prolongation

    Return prolongation operator for use in multi-grid

Range(...)
    Range(self: ngsolve.comp.FESpace, arg0: typing.SupportsInt | typing.Support

    deprecated, will be only available for ProductSpace

SetCouplingType(...)
    SetCouplingType(*args, **kwargs)
    Overloaded function.

    1. SetCouplingType(self: ngsolve.comp.FESpace, dofnr: typing.SupportsInt |

```

Back to top
Set coupling type for degree of freedom.

Parameters:

```

dofnr : int
    input dof number

coupling_type : ngsolve.comp.COUPLING_TYPE
    input coupling type

2. SetCouplingType(self: ngsolve.comp.FESpace, dofnrs: ngsolve.ngstd.IntRange)

    Set coupling type for interval of dofs.

Parameters:

dofnrs : Range
    range of dofs

coupling_type : ngsolve.comp.COUPLING_TYPE
    input coupling type

SetDefinedOn(...)
SetDefinedOn(self: ngsolve.comp.FESpace, region: ngsolve.comp.Region) -> None

    Set the regions on which the FESpace is defined.

Parameters:

region : ngsolve.comp.Region
    input region

SetHarmonicProlongation(...)
SetHarmonicProlongation(self: ngsolve.comp.FESpace, bf: ngcomp::BilinearForm)

    use harmonic prolongation w.r.t. bilinear-form

SetOrder(...)
SetOrder(*args, **kwargs)
Overloaded function.

1. SetOrder(self: ngsolve.comp.FESpace, element_type: ngsolve.fem.ET, order: int)

Parameters:

element_type : ngsolve.fem.ET
    input element type

order : object
    input polynomial order
    Back to top

2. SetOrder(self: ngsolve.comp.FESpace, nodeid: ngsolve.comp.NodeId, order: int)

```

```

Parameters:

nodeid : ngsolve.comp.NodeId
    input node id

order : int
    input polynomial order

SolveM(...)
    SolveM(self: ngsolve.comp.FESpace, vec: ngsolve.la.BaseVector, rho: ngsolve

        Solve with the mass-matrix. Available only for L2-like spaces.

Parameters:

vec : ngsolve.la.BaseVector
    input right hand side vector

rho : ngsolve.fem.CoefficientFunction
    input CF

TestFunction(...)
    TestFunction(self: ngsolve.comp.FESpace) -> object

    Return a proxy to be used as a testfunction for Symbolic Integrators

TnT(...)
    TnT(self: ngsolve.comp.FESpace) -> tuple[object, object]

    Return a tuple of trial and testfunction

TraceOperator(...)
    TraceOperator(self: ngsolve.comp.FESpace, tracespace: ngsolve.comp.FESpace,

TrialFunction(...)
    TrialFunction(self: ngsolve.comp.FESpace) -> object

    Return a proxy to be used as a trialfunction in Symbolic Integrators

Update(...)
    Update(self: ngsolve.comp.FESpace) -> None

    update space after mesh-refinement

UpdateDofTables(...)
    UpdateDofTables(self: ngsolve.comp.FESpace) -> None
        Back to top
    update dof-tables after changing polynomial order distribution

__eq__(...)
    __eq__(self: ngsolve.comp.FESpace, space: ngsolve.comp.FESpace) -> bool

```

```

__mul__(...)
    __mul__(self: ngsolve.comp.FESpace, arg0: ngsolve.comp.FESpace) -> ngcomp::I

__pow__(...)
    __pow__(self: ngsolve.comp.FESpace, arg0: typing.SupportsInt | typing.SupportsComplex) -> ngcomp::I

__str__(...)
    __str__(self: ngsolve.comp.FESpace) -> str

__timing__(...)
    __timing__(self: ngsolve.comp.FESpace) -> object

-----
Static methods inherited from ngsolve.comp.FESpace:

__special_treated_flags__(...) method of pybind11_builtins.pybind11_detail_function
    __special_treated_flags__() -> dict

-----
Readonly properties inherited from ngsolve.comp.FESpace:

autoupdate

components
    deprecated, will be only available for ProductSpace

couplingtype

dim
    multi-dim of FESpace

globalorder
    query global order of space

is_complex

loembedding

loSPACE

mesh
    mesh on which the FESpace is created

ndof
    number of degrees of freedom

ndofglobal
    global number of dofs on MPI-distributed mesh

type
    type of finite element space
    Back to top
-----
Data and other attributes inherited from ngsolve.comp.FESpace:

```

```

__hash__ = None

-----
Readonly properties inherited from ngsolve.comp.NGS_Object:

__memory__

flags

-----
Data descriptors inherited from ngsolve.comp.NGS_Object:

name

-----
Static methods inherited from pybind11_builtins.pybind11_object:

__new__(*args, **kwargs) class method of pybind11_builtins.pybind11_object
    Create and return a new object.  See help(type) for accurate signature.
```

Exercises:

Extend MyFESpace by the following elements:

- 1D finite elements (ET_SEGM), as needed for boundary conditions, P^1 and P^2 .
- quadrilateral elements (ET_QUAD), space Q^1 , use
geom.GenerateMesh(quad_dominated=True)
- tetrahedral elements (ET_TET), P^1 and P^2 , test it for 3D domains

Next, implement P^3 triangles and Q^2 quadrilaterals.

[Back to top](#)